

Detección híbrida de productos en supermercado

Estado del proyecto y guía de incorporación

Visión clásica + Deep Learning · Visión por Computador · UAB

Equipo

Artur Moret González · Daniel Cruz Flores · Marco Mejías Alés

Profesor

Felipe Lumbreras Ruiz

Índice

Mapa rápido del documento. Si solo quieres poner tu máquina al día, salta directamente al capítulo 5.

1.	Resumen ejecutivo	Qué hace el proyecto y dónde estamos
2.	El problema y el enfoque	Qué queremos resolver y cómo
3.	Arquitectura del pipeline	El sistema en una página
4.	Datasets que usamos	Klasson, Freiburg y el futuro Monster
5.	Setup paso a paso	Lo que tienes que ejecutar para tener todo igual que tu compañero
6.	Estructura del repositorio	Qué hay en cada carpeta
7.	Cómo funciona cada bloque del código	Preprocesado, region proposal, features, clasificación
8.	Resultados actuales	Métricas que tenemos a fecha de hoy
9.	Próximos pasos	F4 en adelante: dataset Monster + Deep Learning
10.	Glosario	Siglas y términos por si alguno se te escapa

1. Resumen ejecutivo

Lo que un compañero debe saber en 2 minutos.

Estamos construyendo un sistema de visión por computador que detecta productos en imágenes de supermercado. La asignatura nos exige que no sea una solución 100 % Deep Learning, así que combinamos **técnicas clásicas** (segmentación por color, descriptores HOG y LBP, clasificadores SVM y Random Forest) con un componente acotado de **Deep Learning** (una CNN entrenada sobre fotos propias de bebidas Monster). El resultado final será un pipeline en cascada que, dada una imagen de estantería, devuelve cajas con la categoría del producto y, si es una lata Monster, su variante concreta.

Estado a fecha de hoy

Bloque	Detalle	Estado
Preprocesado	White balance + CLAHE + denoising bilateral	✓ Hecho
Region proposal	Segmentación HSV + morfología + contornos + NMS	✓ Hecho
Features	Histograma HSV + HOG + LBP (1806 dims totales)	✓ Hecho
Clasificación clásica	SVM (con grid search) y Random Forest, comparados	✓ Hecho
Datasets públicos	Klasson (KTH) + Freiburg Groceries combinados	✓ Hecho
Captura del dataset Monster	Sesiones de fotos en supermercados	■ Pendiente
Modelo Deep Learning	CNN con transfer learning sobre fotos Monster	■ Pendiente
Pipeline integrado	Region proposal → SVM → CNN si la categoría es 'lata'	■ Pendiente
Memoria final	Documento académico final con resultados	■ Pendiente

Métricas actuales (parte clásica)

Sobre el dataset combinado Klasson + Freiburg con 8 categorías (fruta, verdura, brick, lata, botella, caja, bolsa, tarro) y ~7 500 imágenes, el SVM con kernel RBF alcanza $\text{accuracy} \approx 0.82$ y $\text{F1 macro} \approx 0.79$ en test. Random Forest queda por detrás ($\text{F1} \approx 0.67$), aunque depende de la última corrida. La cifra es la que tenemos en el momento de escribir esto y se actualizará si reentrenamos.

2. El problema y el enfoque

Qué queremos resolver, por qué es difícil y qué decisiones hemos tomado.

El problema

La detección automática de productos en retail es un problema clásico de visión por computador con aplicación directa: control de stock, detección de huecos en estantería, sistemas de cobro automático, auditoría de planogramas. La dificultad está en las condiciones reales de captura: la iluminación cambia mucho entre tiendas, hay oclusiones, productos pegados, envases reflectantes o transparentes y categorías visualmente parecidas (una manzana verde y un pimiento verde tienen un color similar).

La restricción de la asignatura

Nos obligan a que la solución no sea *full deep learning*. Las redes convolucionales modernas resuelven este problema con relativa facilidad, pero el objetivo del curso es que aprendamos las técnicas clásicas que la mayoría de la gente ya no implementa a mano. El proyecto se diseña entonces como un **pipeline híbrido**: la mayor parte del trabajo lo hacen técnicas clásicas, y el Deep Learning entra solo en una tarea muy acotada.

Nuestro enfoque

El pipeline trabaja en dos fases bien diferenciadas:

- **Fase clásica** — preprocesado, generación de regiones candidatas y clasificación por *categoría* genérica (fruta, lata, botella...). Usa datasets públicos para entrenarse.
- **Fase Deep Learning** — sobre las cajas que la fase clásica etiqueta como 'lata', una CNN entrenada con fotos propias decide si es una lata Monster y, en su caso, qué *variante* (Original, Ultra, Mango Loco...).

Por qué no es full Deep Learning

El grueso del pipeline (preprocesado, segmentación, generación de regiones, descriptores y clasificación de categorías genéricas) se resuelve con técnicas clásicas. La red profunda solo se aplica a una subtarea concreta: la identificación fina del producto Monster.

3. Arquitectura del pipeline

El sistema completo en un solo lugar.

El pipeline procesa una imagen de entrada en cuatro etapas. Las tres primeras son responsabilidad de la visión clásica. La cuarta se divide en una rama clásica (clasificación de categorías) y una rama de Deep Learning (identificación de Monster).

Etapa	Bloque	Técnica principal
1. Preprocesado	Clásico	White balance + CLAHE + denoising bilateral
2. <i>Region proposal</i>	Clásico	Segmentación HSV + morfología + contornos + NMS
3. Extracción de <i>features</i>	Clásico	Histograma HSV + HOG + LBP
4a. Clasificación por categoría	Clásico (ML)	SVM (RBF) y Random Forest, mejor el SVM
4b. Identificación de Monster	Deep Learning	CNN con <i>transfer learning</i> sobre dataset propio

Flujo de una imagen a través del sistema

- **(1)** La imagen entra en el módulo *preprocessing*. Se le corrige el balance de blancos (gray world), se aplica CLAHE sobre el canal L del espacio Lab para igualar el contraste local, y se aplica un filtro bilateral para reducir ruido sin perder bordes.
- **(2)** La imagen preprocesada pasa por *region proposal*. Se convierte a HSV, se generan máscaras binarias por categoría de color (rojo, verde, naranja, etc.), se limpian con morfología, y se extraen *bounding boxes* de los contornos. Cajas demasiado pequeñas, demasiado grandes o con proporciones absurdas se descartan, y un paso de *Non-Maximum Suppression* elimina duplicados.
- **(3)** Para cada caja candidata se recorta la subimagen y se extraen tres descriptores clásicos: histograma HSV (color), HOG (forma) y LBP (textura). Los tres se concatenan en un único vector de 1806 dimensiones.
- **(4a)** Ese vector pasa al SVM (con StandardScaler delante), que predice una de las 8 categorías genéricas: fruta, verdura, brick, lata, botella, caja, bolsa o tarro.
- **(4b)** [PENDIENTE] Si la categoría predicha es 'lata', el *crop* se pasa también a la CNN entrenada con fotos Monster. Esta confirma si es una lata Monster y, si lo es, predice la variante.
- **(5)** La salida es la imagen original con todas las cajas anotadas: cada una con su categoría y, cuando proceda, su etiqueta fina de Monster.

4. Datasets que usamos

De dónde sale cada imagen, por qué la usamos y cuáles son sus límites.

Klasson — Grocery Store Dataset (KTH)

Dataset publicado por Klasson, Zhang y Kjellström en WACV 2019. Contiene unas 5 000 imágenes capturadas en supermercados reales, organizadas en una jerarquía de tres niveles: grupo (Fruit, Packages, Vegetables), clase (Apple, Banana, Juice...) y subclase de la marca concreta (Granny Smith, Royal Gala...). Lo usamos por dos motivos: está alojado en GitHub y se descarga sin problemas, y ofrece imágenes en condiciones realistas de iluminación de tienda.

Cómo lo procesamos. El script `scripts/flatten_klasson.py` aplanar la jerarquía al nivel medio (Apple, Juice...) usando enlaces simbólicos, sin duplicar espacio en disco. El resultado queda en `data/external/klasnon_flat/`.

Freiburg Groceries Dataset

Dataset de Jund, Abdo, Eitel y Burgard (Universidad de Freiburg, 2016). Tiene 25 categorías de productos típicos de supermercado: cereales, café, pasta, snacks, latas, botellas... Es complementario a Klasson, que se queda en frescos y bricks. Sumando los dos cubrimos las 8 categorías que el proyecto necesita.

Por qué fue un poco lío descargarlo. El servidor original (`aisdatasets.cs.uni-freiburg.de`) suele estar caído. El script `scripts/download_freiburg_v2.py` clona el repositorio oficial en GitHub y delega la descarga al script que los propios autores incluyen. Al hacerlo, las imágenes acaban en `data/external/images/` (un nivel arriba de lo esperado) porque el script de los autores extrae con rutas relativas `-C ../`. Tras la descarga hay que mover esa carpeta a su sitio:

```
mv data/external/images data/external/freiburg_groceries/images
```

Dataset combinado

El script `scripts/combine_datasets.py` une Klasson y Freiburg en un único directorio `data/external/combined/` usando enlaces simbólicos. Cada clase de cada dataset queda como una carpeta independiente. El mapeo de cada clase a una de las 8 categorías del proyecto (fruta, verdura, brick, lata, botella, caja, bolsa, tarro) se hace en `src/classification/class_mapping.py`.

Tras la combinación tenemos aproximadamente 7 500 imágenes repartidas entre 68 clases originales y agrupadas en las 8 categorías del proyecto.

El futuro: dataset Monster propio

Para la fase de Deep Learning capturaremos un dataset propio de bebidas energéticas Monster en distintos supermercados. El protocolo está definido en `docs/protocolo_captura.md` e incluye convenciones de nombres, distribución de tipos de foto (lineal completo, primer plano, oclusiones...) y herramientas de anotación. El objetivo son entre 500 y 1 000 imágenes anotadas con *bounding boxes* y etiqueta de variante.

Convenciones importantes

Las fotos crudas no se suben a Git (lo bloquea .gitignore). Para compartirlas usamos un Drive del equipo. A Git solo subimos código, anotaciones, notebooks y documentación.

5. Setup paso a paso

Lo que tienes que ejecutar para llegar exactamente al punto en el que estamos.

Este capítulo es la parte importante para ti si te acabas de incorporar. Sigue los pasos en orden. Si alguno falla, no avances hasta resolverlo: cada paso depende del anterior.

5.1. Requisitos previos

- **Linux o Mac** (en Windows funciona pero no lo hemos probado).
- **Python 3.10–3.13**. Python 3.14 puede dar problemas con algunas librerías; si tu sistema solo tiene 3.14, instala 3.12 con tu gestor de paquetes.
- **git, pip** y un editor (VS Code, PyCharm, lo que prefieras).
- **Espacio en disco**: al menos 4 GB para datasets y modelos.

5.2. Clonar el repositorio

```
git clone https://github.com/marco-mejias/object-detection-cart-on.git
cd object-detection-cart-on
```

5.3. Crear entorno virtual e instalar dependencias

En la raíz del proyecto, con el venv activado:

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install notebook ipykernel
```

Verifica que el venv está activo:

```
which python
# Debe imprimir algo como: ~/proyecto-vc/venv/bin/python
```

Aviso si usas Fedora con Python 3.14

Si `python3 -m venv venv` falla, instala Python 3.12 con `sudo dnf install python3.12 python3.12-devel` y crea el venv con `python3.12 -m venv venv`.

5.4. Descargar Klasson

```
python scripts/download_klasson.py
```

Clona el repositorio oficial con `git clone --depth 1 en data/external/grocery_store/`. Tarda 1-2 minutos.

5.5. Aplanar Klasson

```
python scripts/flatten_klasson.py
```

Crea data/external/klasson_flat/ con una carpeta por clase (Apple, Banana, Juice...). Tardará pocos segundos.

5.6. Descargar Freiburg

```
python scripts/download_freiburg_v2.py
```

Tarda entre 5 y 15 minutos según conexión (~350 MB).

Importante: paso manual tras la descarga

Cuando termine, mueve las imágenes a su ubicación esperada:

```
mv data/external/images data/external/freiburg_groceries/images
```

El comando funciona aunque no exista la carpeta destino. Comprueba con `ls data/external/freiburg_groceries/images | head`.

5.7. Combinar los datasets

```
python scripts/combine_datasets.py
```

Crea data/external/combined/ con todas las clases de ambos datasets. La salida te dirá cuántas imágenes hay y cómo se reparten.

5.8. Construir el dataset de features

Asegúrate de que scripts/build_dataset.py apunta al dataset combinado:

```
# En scripts/build_dataset.py:  
DATA_ROOT = Path("data/external/combined")
```

Después ejecuta:

```
python scripts/build_dataset.py
```

Recorre las ~7 500 imágenes, extrae HSV+HOG+LBP de cada una y guarda X.npy e y.npy en results/dataset/. Tarda 1-3 minutos en máquina normal.

5.9. Entrenar y comparar SVM vs Random Forest

```
python scripts/compare_svm_rf.py
```

Hace *grid search* del SVM probando 9 combinaciones de (*C*, *gamma*) con cross-validation 3-fold, entrena un Random Forest, compara los dos en test, y guarda la imagen comparativa en results/figures/comparacion_svm_rf.png.

Esto tarda

El grid search del SVM con ~5 300 muestras de entrenamiento puede tardar entre 15 y 50 minutos según la CPU. Es normal. Déjalo correr y haz otra cosa.

5.10. Comprobar que todo funciona

Si llegaste hasta aquí, ya estás al día. Para verificarlo:

- `data/external/klasson_flat/` existe y tiene carpetas por clase.
- `data/external/freiburg_groceries/images/` existe y tiene 25 carpetas.
- `data/external/combined/` existe y tiene unas 68 carpetas.
- `results/dataset/X.npy` y `y.npy` están generados.
- `results/models/svm_categoria_gridsearch.joblib` y `rf_categoria.joblib` existen.
- `results/figures/comparacion_svm_rf.png` se ve sin errores.

6. Estructura del repositorio

Mapa de carpetas y para qué sirve cada una.

```
proyecto-vc/
├── data/
│   ├── raw/ # fotos Monster propias (no en Git)
│   ├── processed/ # imágenes ya preprocesadas
│   ├── annotations/ # bounding boxes en YOLO/COCO
│   ├── external/ # datasets públicos descargados
│   ├── grocery_store/ # Klasson (clonado de GitHub)
│   ├── klasson_flat/ # Klasson aplanado (symlinks)
│   ├── freiburg_groceries/ # Freiburg descargado
│   └── combined/ # Klasson + Freiburg (symlinks)
├── docs/
│   └── protocolo_captura.md # cómo capturar fotos Monster
├── notebooks/
│   ├── 01_exploracion_dataset.ipynb
│   ├── 02_preprocesado.ipynb
│   ├── 02b_preprocesado_completo.ipynb
│   ├── 03_region_proposal.ipynb
│   └── 04_evaluacion_svm.ipynb
```

```
■■■ scripts/
■ ■■■ check_environment.py
■ ■■■ rename_photos.py
■ ■■■ dataset_stats.py
■ ■■■ download_klasson.py
■ ■■■ flatten_klasson.py
■ ■■■ download_freiburg_v2.py
■ ■■■ combine_datasets.py
■ ■■■ smoke_test_f2.py / smoke_test_full.py
■ ■■■ build_dataset.py
■ ■■■ train_svm.py
■ ■■■ compare_svm_rf.py
■■■ src/
■ ■■■ preprocessing/ # WB + CLAHE + denoise
■ ■■■ region_proposal/ # HSV + morfología + contornos
■ ■■■ features/ # HSV + HOG + LBP
■ ■■■ classification/ # SVM, RF, mapeo de clases
■ ■■■ deep_learning/ # vacío de momento (F5)
■ ■■■ utils/ # helpers I/O
■■■ results/
■ ■■■ dataset/ # X.npy, y.npy generados
■ ■■■ models/ # SVM y RF entrenados (.joblib)
■ ■■■ figures/ # gráficas, smoke tests
■ ■■■ logs/
■■■ requirements.txt
■■■ .gitignore
■■■ README.md
```

7. Cómo funciona cada bloque del código

Una explicación bloque a bloque, sin entrar en cada línea.

7.1. Preprocesado (src/preprocessing/)

La función `preprocess(image)` aplica tres pasos en orden:

- **White balance (Gray World).** Calcula la media de cada canal RGB y reescala para que las tres medias coincidan. Compensa el tinte de las luces (focos cálidos = más rojo, neones = más azul).
- **CLAHE sobre canal L de Lab.** Mejora el contraste local sin alterar los colores. Trabajamos sobre el canal L (luminosidad) del espacio Lab para que la corrección no manche los tonos.
- **Filtro bilateral.** Reduce ruido de cámara conservando los bordes. A diferencia del gaussiano, no emborrona las siluetas.

Los tres pasos se pueden activar/desactivar con flags (`apply_wb`, `apply_clahe_step`, `apply_denoise`) para experimentar.

7.2. Region proposal (src/region_proposal/)

La función `propose_regions(image)` devuelve una lista de objetos Proposal con coordenadas (x , y , w , h), categoría tentativa y score. Por dentro:

- Convierte la imagen a HSV.
- Para cada rango definido en `hsv_ranges.py` (rojo, verde, naranja, amarillo, azul, morado, rosa, neutro/poco saturado) genera una máscara binaria.
- Aplica *closing* y *opening* morfológicos para limpiar las máscaras.
- Extrae los contornos y calcula sus *bounding boxes*.
- Filtra cajas demasiado pequeñas (< 0.5 % del área) o demasiado grandes (> 50 %), y descarta proporciones absurdas.
- Aplica *Non-Maximum Suppression* con umbral $\text{IoU} = 0.3$ para fusionar cajas que se solapan mucho.

Los rangos HSV son un punto de partida razonable. Cuando integremos el dataset Monster habrá que añadir rangos específicos para los verdes saturados de la lata Original, los blancos brillantes de Ultra, etc.

7.3. Features (src/features/)

La función `extract_features(image)` concatena tres descriptores complementarios en un vector de 1806 dimensiones:

- **Histograma HSV (32 dims):** 16 bins para Hue + 8 para Saturation + 8 para Value. Captura el *color* de la región.
- **HOG (1764 dims):** *Histogram of Oriented Gradients* sobre imagen redimensionada a 128×128 en grayscale, con celdas 16×16 , 9 orientaciones y bloques 2×2 . Captura la *forma* de la región.
- **LBP (10 dims):** *Local Binary Patterns* uniform con radio 1 y 8 vecinos. Captura la *textura* (brillo metálico vs plástico vs cartón vs piel de fruta).

7.4. Clasificación (src/classification/)

Tenemos dos clasificadores entrenados, ambos sobre las features estandarizadas con StandardScaler:

- **SVM (RBF kernel).** *Pipeline* de scikit-learn con StandardScaler + SVC. Hiperparámetros óptimos según grid search: C=10, gamma='scale'. class_weight='balanced' para compensar desbalance.
- **Random Forest.** 300 árboles, sin profundidad máxima, min_samples_leaf=2, class_weight='balanced'. Menos preciso que el SVM en este problema, pero útil para sacar *feature importance* y comparar.

Las 8 categorías del proyecto se definen en class_mapping.py: fruta, verdura, brick, lata, botella, caja, bolsa, tarro. El diccionario KLASSON_TO_CATEGORY_FULL traduce cada clase original de Klasson o Freiburg a una de estas 8 categorías. Si el dataset cambia, hay que actualizar este diccionario.

8. Resultados actuales

Métricas a fecha de la última corrida.

Estos números corresponden a la corrida del SVM con grid search y el Random Forest sobre el dataset combinado Klasson + Freiburg, abarcando ya las 8 categorías del proyecto: fruta, verdura, brick, lata, botella, caja, bolsa y tarro. Total de imágenes procesadas: 7 587, repartidas en 5 310 de entrenamiento, 1 138 de validación y 1 139 de test.

Comparativa SVM vs Random Forest (8 categorías)

Métrica	SVM (C=10, γ =scale)	Random Forest (300 árboles)
Test accuracy	0.655	0.582
Test F1 macro	0.635	0.526
Tiempo entrenamiento	46 min (grid search)	9 s
F1 fruta	0.82	0.81
F1 brick	0.77	0.69
F1 tarro	0.64	0.49
F1 caja	0.63	0.55
F1 verdura	0.61	0.35
F1 bolsa	0.55	0.48
F1 botella	0.55	0.59
F1 lata	0.52	0.26

Lectura de los resultados

El SVM gana con holgura: 0.108 puntos por encima en F1 macro. Es esperable que la accuracy haya bajado respecto a la corrida con 3 categorías (de 0.82 a 0.65) porque el problema es bastante más difícil ahora — más clases significan más oportunidades de equivocarse. Pero el F1 macro se mantiene alto y bastante consistente entre clases, lo que indica que el modelo discrimina razonablemente bien las 8.

Lo que va bien. Fruta lidera con F1 0.82 gracias a sus colores y formas distintivas. Brick (0.77) y caja (0.63) también se reconocen sin grandes problemas porque son rectangulares y tienen *packaging* de colores planos. Tarro (0.64) y verdura (0.61) aguantan razonablemente.

Lo que cuesta más. Lata (0.52), bolsa (0.55) y botella (0.55) son las más difíciles. Las latas son metálicas y reflejan, muy fáciles de confundir con tarros y botellas; las bolsas cambian de forma constantemente (arrugas, dobleces); y las botellas comparten silueta cilíndrica con tarros. Son los casos donde la visión clásica encuentra su techo natural.

El comportamiento del Random Forest sigue el mismo patrón conservador que vimos con 3 categorías: en lata tiene precision 0.94 pero recall 0.15, en verdura precision 1.00 con recall 0.21. Cuando se atreve a clasificar acierta, pero la mayoría del tiempo no se atreve. Por eso el SVM, más equilibrado, queda como clasificador principal del proyecto.

Para la memoria

El techo en torno al 65 % accuracy con 8 categorías es consistente con la literatura de visión clásica sobre productos de supermercado. Este nivel justifica la introducción del componente de Deep Learning para identificación fina (Monster). No es un fallo del enfoque clásico: es exactamente la motivación del pipeline híbrido.

9. Próximos pasos

Lo que queda por hacer y en qué orden.

Fase	Tarea	Quién
F4	Capturar dataset Monster (500-1000 fotos en distintas tiendas) siguiendo el protocolo en docs/protocolo_captura.md.	Equipo
F4	Anotar cada foto con bounding boxes + variante usando LabelImg o CVAT. Formato YOLO o COCO.	Equipo
F4	Hacer splits train/val/test 70/15/15 estratificados por escena.	Equipo
F5	Entrenar CNN ligera (MobileNetV2 o YOLOv8n) sobre el dataset Monster con transfer learning.	Equipo
F6	Integrar el pipeline completo: region proposal → SVM → CNN si categoría == 'lata'.	Equipo
F6	Evaluar con métricas end-to-end: mAP@0.5, mAP@0.5:0.95, tiempo de inferencia.	Equipo
F7	Análisis de errores y galería de aciertos / fallos.	Equipo
F7	Memoria final con resultados, comparativas y conclusiones.	Equipo

Decisiones todavía pendientes

- **Variantes de Monster a incluir.** Plan inicial: 3 variantes (Original, Ultra, Mango Loco) por disponibilidad. Ampliar si se encuentran las demás.
- **Modelo de Deep Learning.** Decidir entre dos opciones: clasificador sobre *crops* (CNN ligera que opera sobre las cajas de la parte clásica) o detector end-to-end (YOLOv8n entrenado de cero). El primero es más coherente con el espíritu híbrido del proyecto.
- **Plataforma de entrenamiento.** Por ahora suponemos Google Colab (GPU T4 gratis). Si el dataset crece, plantearse Colab Pro o un ordenador con GPU dedicada.

10. Glosario

Siglas y términos que aparecen mucho.

CLAHE	Contrast Limited Adaptive Histogram Equalization. Mejora el contraste local de la imagen sin amplificar ruido en exceso.
HSV	Espacio de color: Hue (tono), Saturation (saturación), Value (luminosidad). Más robusto a cambios de luz que RGB.
Lab	Espacio de color: L (luminosidad), a y b (canales de color). Lo usamos para aplicar CLAHE solo al canal L.
HOG	Histogram of Oriented Gradients. Descriptor que captura siluetas y bordes.
LBP	Local Binary Patterns. Descriptor de textura: para cada píxel, codifica si sus vecinos son más claros o más oscuros.
NMS	Non-Maximum Suppression. Elimina cajas duplicadas que se solapan demasiado.
IoU	Intersection over Union. Mide cuánto se solapan dos cajas (0 = nada, 1 = idénticas).
SVM	Support Vector Machine. Clasificador clásico que busca el hiperplano óptimo de separación entre clases.
RBF	Radial Basis Function. Kernel del SVM que permite separar clases no lineales.
F1 macro	Métrica de clasificación que promedia el F1-score por clase, dándole el mismo peso a cada una. Útil con datasets desbalanceados.
mAP	Mean Average Precision. Métrica estándar en detección de objetos. Combina precision y recall a varios umbrales.
Brick	Envases de cartón típicos del lácteo y zumo (1L). En nuestro proyecto es una de las 8 categorías.
Region proposal	Etapa que genera cajas candidatas donde podría haber productos, sin clasificarlos todavía.
Transfer learning	Técnica de Deep Learning donde se reutiliza un modelo preentrenado (en ImageNet, etc.) y se ajusta a una tarea concreta con pocos datos.

¿Algo no cuadra?

Pregunta a Artur.